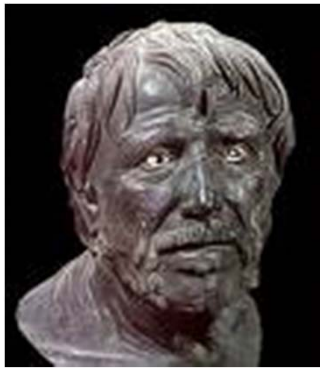
“No nos hace falta valor para emprender ciertas cosas porque sean difíciles, sino que son difíciles porque nos falta valor para emprenderlas”.

Lucio Anneo Séneca.

Córdoba año 4 a. C. - Roma año 65.



Diseño con Microcontroladores.

Tema 3: ***Software del microcontrolador M16C/62 de Renesas***
(4 horas + 8 horas de laboratorio)

3.1. Modelo de programación: Lenguaje ensamblador.

3.2. Instrucciones básicas.

3.3. Modos de direccionamiento.

3.4. Gestión de direcciones.

3.5. Directivas y funciones macro.

3.6. Ejemplo de programa fuente.

3.7. Sistemas de desarrollo 3DKM16C/62PU

Práctica de laboratorio: Manejo del depurador del sistema de desarrollo.

Práctica de laboratorio: Análisis del mini-emulador.

Práctica del laboratorio: Realización de subrutinas. Operaciones con 32 bits.

Tema 3. Software del Microcontrolador M16C/6X

Objetivos específicos:

- Gestionar correctamente el mapa de memoria.

- Manejar el juego de instrucciones básicas:

  transferencia de datos, operaciones aritméticas y lógicas, control, ...

- Describir la secuencia de instrucciones y los conceptos

  de subrutina e interrupción

- Explicar los distintos modos de direccionamiento disponibles y su notación.

- Conocer la gestión de direcciones y el uso de directivas y macros.

- Analizar y utilizar un sistema de desarrollo: ensamblador, enlazador,

  depurador y mini-emulador.

Tema 3. Software del Microcontrolador M16C/6X

Bibliografía recomendada. http://www.renesas.com/

[Mitsubishi, 1998]. Para este tema es fundamental la lectura del manual software suministrado por el fabricante.

[Mitsubishi, 2000a, 2000b, 2000c]. Es necesario conocer y consultar el manual del depurador y el manual de usuario del mini-emulador.

y las notas de aplicación sobre los que se desarrollarán todas las prácticas de laboratorio.

[Cady, 1997b]. Es muy recomendable su manejo por la similitud del ensamblador de Mitsubishi con el de Motorola y por la gran cantidad de programas ejemplos.

Tratamiento didáctico específico.

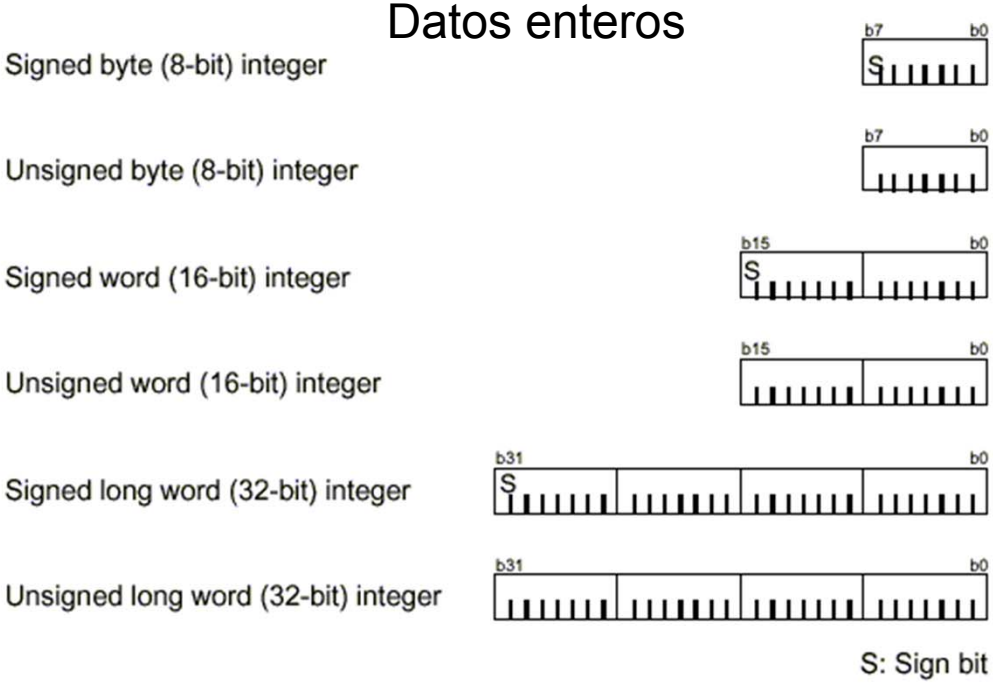
Sobre todo se hará hincapié en el laboratorio de las limitaciones de su uso en cuanto a pines, funciones de los periféricos, restricciones de las interrupciones y demás especificaciones.

Las dos primeras prácticas tendrán tratamiento de *práctica dirigida* y la tercera de *práctica guiada*.

3.1. Modelo de programación. Lenguaje ensamblador

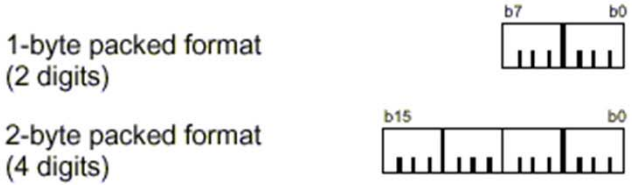
1.- Espacio de direcciones
2.- Conjuntos de registros
3.- Tipos de datos:

Datos enteros



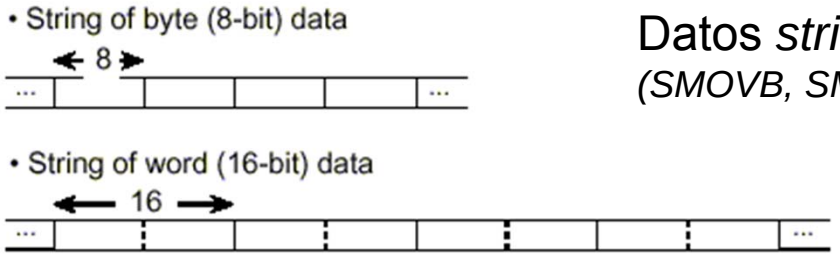
Datos decimales

(DADC, DADD, DSBB y DSUB)



Datos *string*

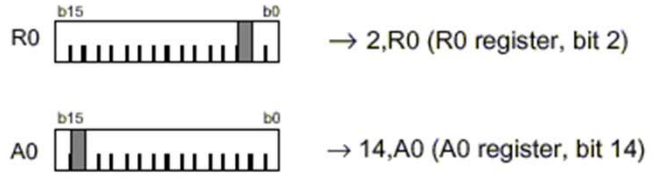
(SMOVB, SMOVF, y SSTR)



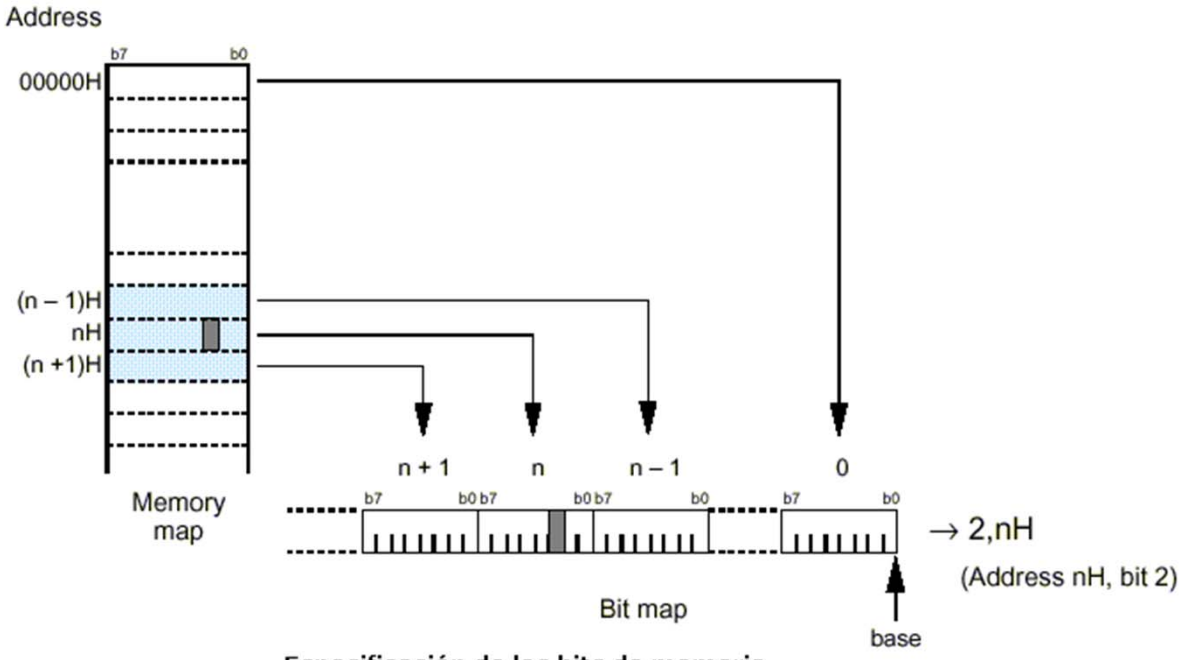
3.1. Modelo de programación. Lenguaje ensamblador

BITS DE REGISTRO Y DE MEMORIA

(14 clases de instrucciones, entre las que se incluyen BCLR, BSET, BTST y BNTST)



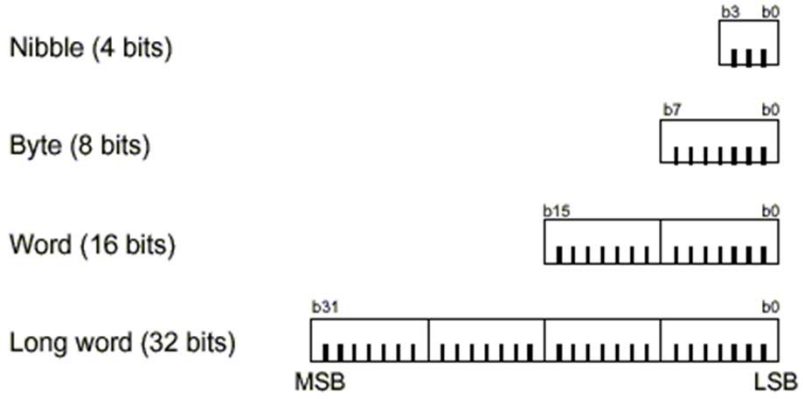
Especificación de los bits de un registro



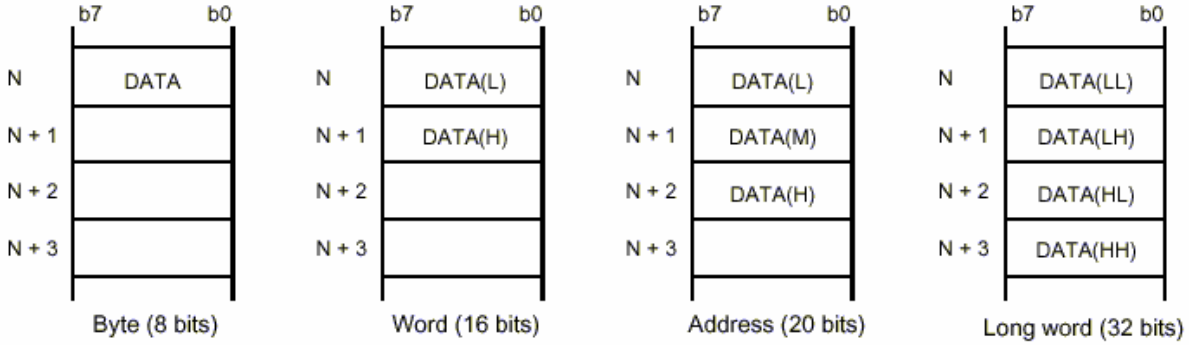
Especificación de los bits de memoria

3.1. Modelo de programación. Lenguaje ensamblador

Disposición de datos en registros



Disposición de datos en memoria



3.1. Modelo de programación. Lenguaje ensamblador

Descripción de los operandos y los operadores

Los operandos se clasifican en 5 tipos:

- **Numéricos**, con los tipos binario, octal, hexadecimal y decimal y se especifican escribiendo al final del número una B, O, H y ningún carácter, respectivamente. Pueden ser usadas las minúsculas: b, o, h. Hay que notar que si en hexadecimal el valor del operando empieza con un carácter alfabético se debe añadir 0 al principio.

- **Nombres**, que se especifican directamente escribiendo el nombre para designar a un símbolo o a una etiqueta.

- **Número en Punto Flotante,** que se especifican con “E” o “e” para la parte exponente. Sólo pueden ser usados en los operandos de las directivas **.FLOAT** (longitud 32-bits) y **.DOUBLE** (longitud 64 bits).

- **Expresión,** que usa una combinación de valores numéricos, en punto flotante, nombres y operadores para especificarlos.

- **Cadena de Caracteres (String),** que se usa encerrando entre comillas simples o dobles comillas la cadena de caracteres. Usa el código ASCIIde 7-bits.

3.1. Modelo de programación. Lenguaje ensamblador

Los operadores se clasifican en 4 tipos:

- **Operadores Unitarios:** + que indica valor positivo, ─ valor negativo, ~ NOT, SIZEOF el tamaño de una sección en bytes, y TOPOF para indicar la dirección de comienzo de la sección.

- **Operadores Binarios**: + que indica suma, ─ resta, * multiplicación, / división, % resto, >> desplazamiento de bits a la derecha, << desplazamiento de bits a la izquierda, & operador lógico AND, | operador lógico OR, ^ operador lógico XOR.

- **Operadores Condicionales**: > mayor que, < menor que, >= mayor o igual que, <= menor o igual que, == igual que, ¡= distinto que. Los operadores condicionales sólo se pueden usar en las directivas .**IF** y **.ELIF**.

- **Operador ()**: se usa para dar prioridad a la hora de calcular una expresión. El término encerrado entre paréntesis es el que tiene la más alta prioridad de ser calculado, después los operadores unitarios, los binarios ((*, /, %), (+, ─), (>>, <<), &, (|, ^), en ese orden y de forma descendente) y por último y con la menor prioridad, los operadores condicionales. Los paréntesis pueden estar concatenados.

3.1. Modelo de programación. Lenguaje ensamblador

Bloques ensamblador:

1.- Cabecera ;comentarios

2.- Definiciones *VramTOP* .EQU 000400H ;comentario

3.- Programa Etiqueta: Instrucción ;comentario

Código de operación fuente(src), destino(dest)

4.- Final .end

Formatos: Genérico [:G]; rápido [:Q]; corto [:S]; cero [:Z]

que se colocan al final del código de operación, sin dejar ningún espacio en blanco, e indican el número de bytes que ocupan en cada instrucción el código de operación y los campos fuente y destino.

3.1. Modelo de programación. Lenguaje ensamblador

Compilación del programa

El programa en ensamblador debe ser escrito en un procesador de texto que use código ASCII, como el Bloc de Notas o el WordPad de Windows. Una vez realizado el código éste se debe salvar con la extensión **.a30**. En el directorio c:\MTOOL\BIN se encuentran los ficheros necesarios para el ensamblado y enlazado del código en ensamblador.

Ensamblado

- Para ensamblar nuestro código se usa el fichero ejecutable **AS30,** El **AS30** es un ensamblador reubicable. Los archivos que genera dicho ensamblador son los siguientes:
- Módulo de fichero reubicable **(.r30)**: Este archivo está basado en el estándar IEEE-695. Contiene los datos en lenguaje máquina y su reubicación.
- Archivo de listado en ensamblador **(.lst)**: Este archivo contiene un listado de las líneas, información de localización, código objeto, e información sobre dichas líneas. Este fichero sólo se genera si se especifica la opción –L.
- Archivo de errores de ensamblador **(.tag)**: Este fichero contiene los mensajes de error que ocurrieron cuando se ensamblaba el fichero fuente. Este fichero no se genera cuando no se encuentra ningún error. Su utilidad es para localizar y poder corregir los errores fácilmente. Este fichero sólo se genera si se especifica la opción –T.

- La sintaxis para ensamblar un fichero fuente es la siguiente:

as30 nombre_fichero.extension [nombre_fichero.extension] [opciones]

3.1. Modelo de programación. Lenguaje ensamblador

Enlazado (Link)

El fichero generado en la fase de ensamblado (practica1.r30) es el que se usa para el enlazado. Para enlazar el código, se utiliza el fichero ejecutable **ln30**. Los archivos que genera el enlazador son:

- **Archivo de módulo absoluto** (.**x30**): Este archivo está basado en el estándar IEEE-695. Consiste en unir en un único archivo varios archivos de módulo reubicables.
- **Archivo de mapas (.map):** Este archivo contiene información de enlazado, información de las direcciones de las secciones, e información de símbolos. Este archivo sólo se genera si se especifica la opción –M ó –MS.
- **Archivo de errores del linkado (.tag):** Este archivo contiene los mensajes de error que se encontraron mientras se enlazaban los archivos de módulos reubicables. Este archivo no se genera si no se encuentran errores. Este archivo permite encontrar y corregir errores fácilmente. Este archivo sólo se genera si se especifica la opción –T.

- La sintaxis para enlazar un fichero de módulo reubicable es la siguiente:

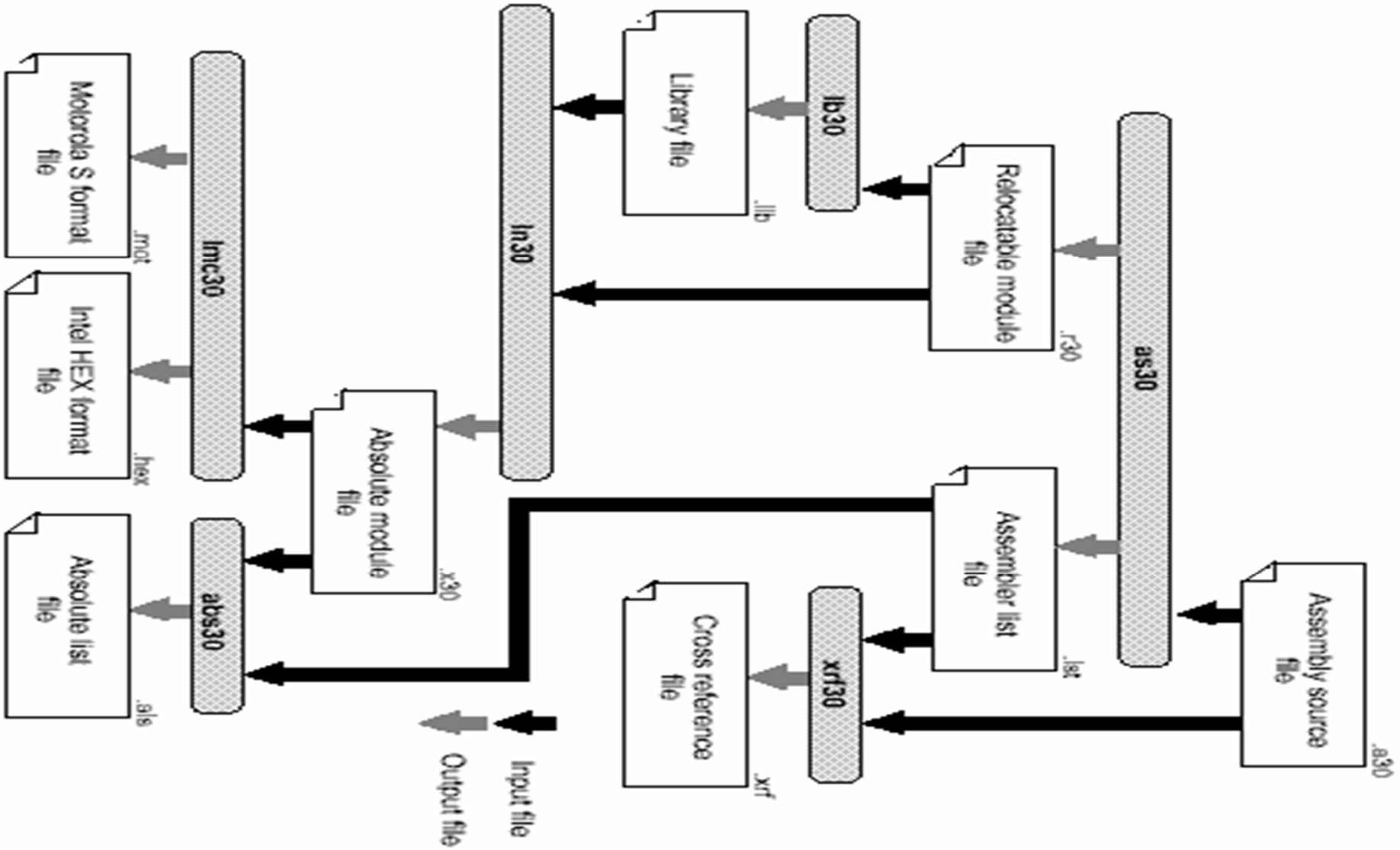
ln30 nombre_fichero_realojable [nombre_fichero_realojable] [opciones]

- Si el fichero tiene como extensión **.r30**, no es necesaria introducir la extensión. Entre las opciones , las más interesantes son la que genera el archivo de mapas (.map), introduciendo la opción *–M*, o la opción –MS para incluir la información de los símbolos, y la que genera el archivo de error de linkado (.tag), introduciendo la opción –T. La opción –G es muy interesante porque permite ver el fichero fuente tal como se escribió desde el depurador.12
- Por ejemplo, para enlazar el fichero practica1.r30, se debe escribir lo siguiente:

ln30 practica1 –MS –T –G

- Este comando genera el fichero practica1.x30 y es el que se utiliza en el depurador KD30

3.1. Modelo de programación. Lenguaje ensamblador



Quick Reference by Function

Function	Mnemonic	Content	See page for function	See page for instruction code /number of cycles
Transfer	MOV	Transfer	92	195
	MOVA	Transfer effective address	94	202
	MOVDir	Transfer 4-bit data	95	203
	POP	Restore register/memory	103	213
	POPM	Restore multiple registers	105	215
	PUSH	Save register/memory/immediate data	106	216
	PUSHA	Save effective address	107	218
	PUSHM	Save multiple registers	109	219
	LDE	Transfer from extended data area	89	193
	STE	Transfer to extended data area	125	235
	STNZ	Conditional transfer	126	237
	STZ	Conditional transfer	127	237
	STZX	Conditional transfer	128	238
	XCHG	Exchange	134	244
Bit manipulation	BAND	Logically AND bits	47	152
	BCLR	Clear bit	48	152
	BM*Cnd*	Conditional bit transfer	49	154
	BNAND	Logically AND inverted bits	50	155
	BNOR	Logically OR inverted bits	51	156
	BNOT	Invert bit	52	156
	BNTST	Test inverted bit	53	157
	BNXOR	Exclusive OR inverted bits	54	158
	BOR	Logically OR bits	55	158
	BSET	Set bit	57	159
	BTST	Test bit	58	160
	BTSTC	Test bit & clear	59	161
	BTSTS	Test bit & set	60	162
	BXOR	Exclusive OR bits	61	162
Shift	ROLC	Rotate left with carry	112	220
	RORC	Rotate right with carry	113	221
	ROT	Rotate	114	222
	SHA	Shift arithmetic	118	227
	SHL	Shift logical	119	230
Arithmetic	ABS	Absolute value	39	140
	ADC	Add with carry	40	140
	ADCF	Add carry flag	41	142
	ADD	Add without carry	42	142
	CMP	Compare	62	163
	DADC	Decimal add with carry	64	167

Quick Reference by Function

Function	Mnemonic	Content	See page for function	See page for instruction code /number of cycles
Arithmetic	DADD	Decimal add without carry	65	169
	DEC	Decrement	66	171
	DIV	Signed divide	67	172
	DIVU	Unsigned divide	68	173
	DIVX	Singed divide	69	174
	DSBB	Decimal subtract with borrow	70	175
	DSUB	Decimal subtract without borrow	71	177
	EXTS	Extend sign	74	180
	INC	Increment	77	182
	MUL	Signed multiply	96	205
	MULU	Unsigned multiply	97	207
	NEG	Two's complement	98	209
	RMPA	Calculate sum-of-products	111	220
	SBB	Subtract with borrow	116	224
	SUB	Subtract without borrow	129	238
Logical	AND	Logical AND	45	149
	NOT	Invert all bits	100	210
	OR	Logical OR	101	211
	TST	Test	131	241
	XOR	Exclusive OR	135	245
Jump	ADJNZ	Add & conditional jump	44	148
	SBJNZ	Subtract & conditional jump	117	226
	JCnd	Jump on condition	80	184
	JMP	Unconditional jump	81	185
	JMPI	Jump indirect	82	187
	JMPS	Jump to special page	83	188
	JSR	Subroutine call	84	189
	JSRI	Indirect subroutine call	85	190
	JSRS	Special page subroutine call	86	191
	RTS	Return from subroutine	115	223
String	SMOVB	Transfer string backward	120	232
	SMOVF	Transfer string forward	121	233
	SSTR	Store string	122	233
Other	BRK	Debug interrupt	56	159
	ENTER	Build stack frame	72	179
	EXITD	Deallocate stack frame	73	180
	FCLR	Clear flag register bit	75	181
	FSET	Set flag register bit	76	182
	INT	Interrupt by INT instruction	78	183
	INTO	Interrupt on overflow	79	184
	LDC	Transfer to control register	87	191

3.2. Instrucciones básicas

Quick Reference by Function

Function	Mnemonic	Content	See page for function	See page for instruction code /number of cycles
Other	LDCTX	Restore context	88	192
	LDINTB	Transfer to INTB register	90	194
	LDIPL	Set interrupt enable level	91	195
	NOP	No operation	99	209
	POPC	Restore control register	104	215
	PUSHC	Save control register	108	218
	REIT	Return from interrupt	110	219
	STC	Transfer from control register	123	234
	STCTX	Save context	124	235
	UND	Interrupt for undefined instruction	132	243
	WAIT	Wait	133	243

Quick Reference by Addressing (general instruction addressing)

Mnemonic	Addressing															See page for function	See page for instruction code /number of cycles
	R0L/R0	R0H/R1	R1L/R2	R1H/R3	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB]	abs16	#IMM8	#IMM16	#IMM20	#IMM		
ABS	○	○	○	○	○	○	○	○	○	○	○					39	140
ADC	○	○	○	○	○	○	○	○	○	○	○	○	○			40	140
ADCF	○	○	○	○	○	○	○	○	○	○	○					41	142
ADD*1	○	○	○	○	○	○	○	○	○	○	○	○	○			42	142
ADJNZ*1	○	○	○	○	○	○	○	○	○	○	○				○	44	148
AND	○	○	○	○	○	○	○	○	○	○	○	○	○			45	149
CMP	○	○	○	○	○	○	○	○	○	○	○	○	○			62	163
DADC	○	○										○	○			64	167
DADD	○	○										○	○			65	169
DEC	○	○			○			○			○					66	171
DIV	○	○	○	○	○	○	○	○	○	○	○	○	○			67	172
DIVU	○	○	○	○	○	○	○	○	○	○	○	○	○			68	173
DIVX	○	○	○	○	○	○	○	○	○	○	○	○	○			69	174
DSBB	○	○										○	○			70	175
DSUB	○	○										○	○			71	177
ENTER												○				72	179
EXTS	○		○*2			○	○	○	○	○	○					74	180
INC	○*3	○*4			○			○			○					77	182
INT															○	78	183
JMPI*1	○	○	○	○	○	○	○	○		○	○					82	187
JMPS												○				83	188
JSRI*1	○	○	○	○	○	○	○	○		○	○					85	190
JSRS												○				86	191
LDC*1	○	○	○	○	○	○	○	○	○	○	○		○			87	191
LDE*1	○	○	○	○	○	○	○	○	○	○	○					89	193

*1 Has special instruction addressing.

*2 Only R1L can be selected.

*3 Only R0L can be selected.

*4 Only R0H can be selected.

ADJNZ

Add & conditional jump

ADdition then Jump on Not Zero

ADJNZ

[Syntax]

ADJNZ.size src,dest,label

B , W

[Instruction Code/Number of Cycles]

Page= 148

[Operation]

dest ← dest + src

if dest ≠ 0 **then** jump label

[Function]

- This instruction adds *dest* and *src* together and stores the result in *dest*.
- If the addition resulted in any value other than 0, control jumps to **label**. If the addition resulted in 0, the next instruction is executed.
- The op-code of this instruction is the same as that of SBJNZ.

[Selectable src/dest/label]

src	dest			label
#IMM*1	R0L/R0 R1H/R3 [A0] dsp:8[A1] dsp:16[A0] abs16	R0H/R1 A0/A0 [A1] dsp:8[SB] dsp:16[A1]	R1L/R2 A1/A1 dsp:8[A0] dsp:8[FB] dsp:16[SB]	PC*2–126 ≦ label ≦ PC*2+129

*1 The range of values that can be taken on is –8 ≤ #IMM ≤ +7.

*2 PC indicates the start address of the instruction.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	–	–	–	–	–	–	–	–

[Description Example]

```
ADJNZ.W  #–1,R0,label
```

[Related Instructions] SBJNZ

JCnd

Jump on condition
Jump on Condition

JCnd

[Syntax]

JCnd **label**

[Instruction Code/Number of Cycles]

Page= 184

[Operation]

if true then jump label

[Function]

- This instruction causes program flow to branch off after checking the execution result of the preceding instruction against the following condition. If the condition indicated by *Cnd* is true, control jumps to **label**. If false, the next instruction is executed.
- The following conditions can be used for *Cnd*:

Cnd	Condition		Expression	*Cnd*	Condition		Expression
GEU/C	C=1	Equal to or greater than C flag is 1.	≦	LTU/NC	C=0	Smaller than C flag is 0.	>
EQ/Z	Z=1	Equal to Z flag is 1.	=	NE/NZ	Z=0	Not equal Z flag is 0.	≠
GTU	C∧$\overline{Z}$=1	Greater than	<	LEU	C∧$\overline{Z}$=0	Equal to or smaller than	≧
PZ	S=0	Positive or zero	0≦	N	S=1	Negative	0>
GE	S∀O=0	Equal to or greater than (signed value)	≦	LE	(S∀O)∨Z=1	Equal to or smaller than (signed value)	≧
GT	(S∀O)∨Z=0	Greater than (signed value)	<	LT	S∀O=1	Smaller than (signed value)	>
O	O=1	O flag is 1.		NO	O=0	O flag is 0.	

[Selectable label]

label	*Cnd*
PC*1–127 ≦ label ≦ PC*1+128	GEU/C,GTU,EQ/Z,N,LTU/NC,LEU,NE/NZ,PZ
PC*1–126 ≦ label ≦ PC*1+129	LE,O,GE,GT,NO,LT

*1 PC indicates the start address of the instruction.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	–	–	–	–	–	–	–	–

[Description Example]

```
JEQ     label
JNE     label
```

[Related Instructions] BM*Cnd*

3.2. Instrucciones básicas

Instrucciones de Transferencia

Transferencia de 4-bits: Transfiere los 4 bits de orden superior o inferior de un registro o memoria de 8 bits. Es muy útil para generar código BCD desempaquetado o para programar los puertos de E/S. Tiene la restricción de que siempre o fuente (*src*) o destino (*dest*) debe ser R0L.

Formato: **MOVDir src,dest**

Transferencia Condicional (*Conditional Store*): En esta transferencia se usa el estado del flag Z como la condición de transferencia. Siempre se ha de usar *src* como un direccionamiento inmediato de 8-bits #IMM.

Formato: **STZ src,dest** Transfiere src a dest si Z=1.

STNZ src,dest Transfiere src a dest si Z=0.

STZX src1,src2,dest Transfiere src1 a dest si Z=1 o src2 a dest si Z=0.

Transferencia de cadenas (*string*): Transfiere datos en cadena (colectivamente). Es usada para transferir bloques y para limpiar la memoria RAM (transfiriendo bloques de ceros). Los datos se pueden transferir en bloques de 1 byte (.B) o de 1 palabra (.W), hacia delante (*forward*) o hacia atrás (*back*). Es necesario para aplicar estas instrucciones que previamente se hayan definido una dirección de memoria dode se encuentra la fuente a transferir (en R1H,A0) o un valor de registro (en R0), una dirección de memoria de destino desde donde se empieza a realizar la transferencia (en A1), y una cuenta de transferencia, indicando la cantidad de veces que se repite la transferencia (en R3).

Formato: **SMOVF(.B) (.W)** transfiere cadenas en direcciones ascendentes en bloques de 1 byte (.B) o de 1 palabra (.W), repitiendo el proceso tantas veces como indique el contenido de R3.

SMOVB(.B) (.W) igual al anterior pero de forma descendente. **SSTR(.B) (.W)** almacena una cadena en direcciones incrementales, en cuyo caso, no se necesita la dirección fuente (R1H,A0). Esta es la instrucción usada para limpiar la RAM.

3.2. Instrucciones básicas

Instrucciones Aritméticas (31)

Multiplicación: Se distingue la multiplicación con signo (complemento a dos) o sin signo, especificando el tamaño deseado: (.B) para multiplicaciones de (8x8)-bits = 16-bits; y (.W) para multiplicaciones de (16x16)-bits = 32-bits. En las operaciones de multiplicación no se alteran los estados de los flags. La principal restricción es que si se especifica (.B) no se pueden usar ni en fuente ni en destino los registros de direcciones.

Formato: **MUL(.B) (.W) src,dest** Multiplicación con signo, el producto de src x dest se almacena en dest,

MULU(.B) (.W) src,dest Multiplicación sin signo.

División: Se distingue la división con signo (complemento a dos) o sin signo, especificando el tamaño deseado: (.B) para divisiones de (16/8)-bits = cociente en 8-bits y resto en 8-bits; y (.W) para divisiones de (32x16)-bits = cociente en 16-bits y resto en 16 bits. En las operaciones de división se altera el estado del flag O. La principal restricción es que si se especifica (.B) no se pueden usar ni en fuente ni en destino los registros de direcciones.

DIV(.B) (.W) src División con signo, en el que el resto tiene el mismo signo que el del dividendo, sin alterar el signo del cociente. Si el tamaño especificado es (.B) se divide el contenido de R0 (16-bits) entre *src* (8-bits) y el resultado o cociente se guarda en R0L (8-bits) y el resto en R0H (8-bits). Si el tamaño especificado es (.W) se divide el contenido de R2R0 (32-bits) entre *src* (16-bits) y el cociente se guarda en R0 (16-bits) y el resto en R2 (16-bits).

DIVX(.B) (.W) src División con signo, exactamente igual a la instrucción anterior, con la única excepción de que ahora el resto tiene el mismo signo que el divisor.

DIVU(.B) (.W) src División sin signo, con el mismo formato que las de signo.

Suma en Decimal: Hay dos instrucciones de suma decimal, una con el flag de acarreo C y otra sin acarreo. El estado de los flags S, Z y C pueden cambiar.

DADD(.B) (.W) src,dest Suma en decimal *src* + *dest* sin acarreo y se guarda en dest.

DADC(.B) (.W) src,dest Suma en decimal *src* + *dest* + *C* y se guarda en dest.

Resta en Decimal:

DSUB(.B) (.W) src,dest Resta en decimal *src* - *dest* sin acarreo y se guarda en dest.

DSBB(.B) (.W) src,dest Resta en decimal *src* - *dest* - *C(negado)* y se guarda en dest.

Suma (Resta) y Bifurcación Condicional: Los valores sumados o restados por esta instrucción se limitan a 4-bits inmediatos #IMM (-8 a +7 para suma y -7 a +8 para resta). El rango de direcciones que se pueden saltar es de – 126 a +129 desde la dirección de comienzo de la instrucción ADJNZ/SBJNZ (indicado por el contador de programa PC).

ADJNZ(.B) (.W) #IMM,dest,label Suma #IMM a dest y salta a etiqueta si el resultado de dicha operación (dest) es distinto de cero.

SBJNZ(.B) (.W) #IMM,dest,label Resta #IMM a dest y salta a etiqueta si el resultado de dicha operación (dest) es distinto de cero.

Suma de Productos: Calcula la suma de productos y provoca una interrupción de *overflow* si se ha producido un desbordamiento. El flag O se pone a 1 en cuanto se excede de - 32768 o +32767 para (.B) o de – 2147483648 o + 2147483647 para (.W), incluso antes de terminar el cálculo. Se calcula la suma de productos usando A0 como la dirección del multiplicando, A1 como la dirección del multiplicador y R3 como la operación de cuenta. El resultado del cálculo se almacena en R0 para (.B) y en R2R0 para (.W).

RMPA(.B) (.W) Calcula la suma de productos.

3.2. Instrucciones básicas

Instrucciones de Bit (14)

- **Manipulación de bit a nivel lógico**: Se realiza a nivel de bit para las operaciones lógicas AND, NAND, OR, NOR, XOR, XOR. El resultado de la operación siempre se almacena en el flag C.
  - **BAND src** src & C
  - **BNAND src** src(negado) & C
  - **BOR src** src | C
  - **BNOR src** src (negado) | C
  - **BXOR src** src ^ C
  - **BNXOR src** src (negado) ^ C

- **Transferencia de Bit Condicional:** Se transfiere un 1 si una determinada condición es verdadera, o un 0 en caso de ser falsa. Siempre se usa un flag para determinar si la condición es verdadera o falsa.

EQ/Z	igual (Z es 1)	=
NE/NZ	no igual (Z es 0)	≠
GTU	mayor que (C es 1 y Z es 0)	>
GT	mayor que, con signo (S es 1 y O es 1 y Z es 0) o (S es 0 y O es 0 y Z es 0)	>
GEU/C	mayor o igual que (C es 1)	≥
GE	mayor o igual que, con signo (S es 1 y O es 1) o (S es 0 y O es 0)	≥
LTU/NC	menor que (C es 0)	<
LT	menor que , con signo (S es 1 y O es 0) o (S es 0 y O es 1)	<
LEU	menor o igual que (C es 0 o Z es 1=0)	≤
LE	menor o igual que, con signo (Z es 1) o (S es 1 y O es 0) o (S es 0 y O es 1)	≤
PZ	positivo o cero (S=0)	0≥
N	negativo (S=1)	0<
O	flag O=1	
NO	flag O=0	

3.3. Modos de direccionamiento

MODOS DE DIRECCIONAMIENTO

Los modos de direccionamiento son las distintas formas que puede usar el programador para indicar a la CPU dónde se puede leer un dato, dónde se puede almacenar, o dónde se encuentra la siguiente dirección de memoria con la que se debe operar, etc. La CPU debe obtener siempre la dirección efectiva, es decir la dirección de memoria en la que se encuentra el operando de la instrucción a ejecutar.

. **Direccionamiento de Instrucciones Generales**. Se accede a un área de direcciones entre 00000H y 0FFFFH. Hay 8 tipos, que suelen coincidir en su forma con los de los demás fabricantes de microcontroladores: inmediato, absoluto, directo a registro, indirecto a registro, relativo a registro, relativo a SB, relativo a FB y relativo al puntero de pila.

- **Direccionamiento de Instrucciones Especiales**. Se accede al área completa de direcciones, entre 00000H y FFFFFH.

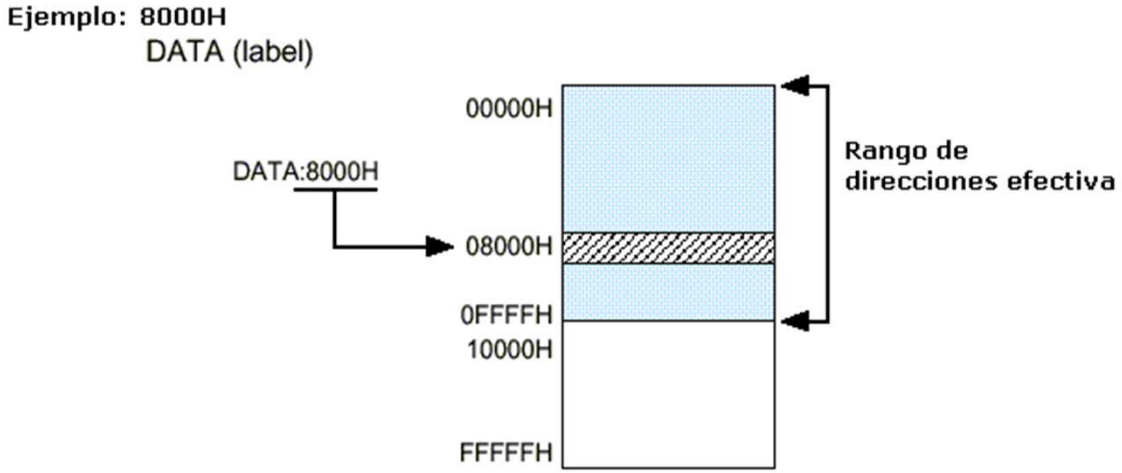
- **Direccionamiento de Instrucciones de Bits**. Se accede a un área de direcciones entre 00000H y 0FFFFH. Hay 7 tipos que coinciden con los de direccionamiento de instrucciones generales (excepto el tipo inmediato), pero a nivel de bit.

3.3. Modos de direccionamiento

Inmediato: El valor inmediato indicado por #IMM es el valor usado directamente para la operación. Es imprescindible añadir el símbolo # al valor inmediato.

Ejemplo: #123 (decimal)
#07DH (hexadecimal)
#01111011B (binario)

Absoluto: El valor indicado es la dirección de memoria efectiva sobre la cual se representa la operación. El rango de direcciones efectivas es 00000_H a $0FFFF_H$.



3.3. Modos de direccionamiento

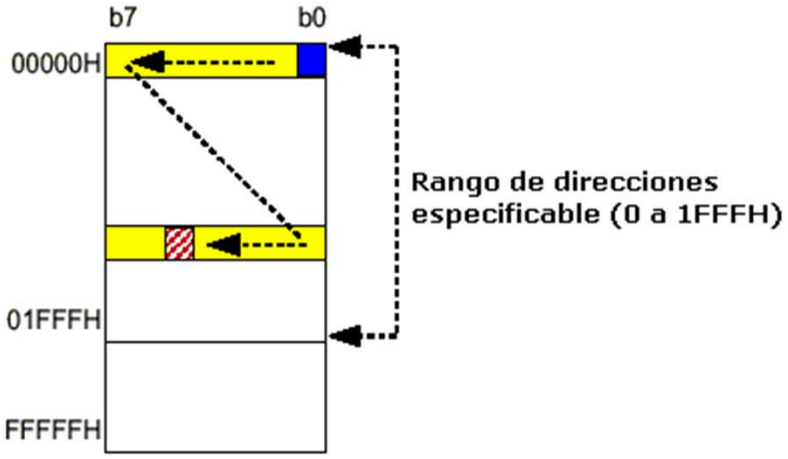
Directo a Registro: Un registro especificado es el dato con el que la operación se representa. Sin embargo, sólo se pueden usar los registros de datos y direcciones.

Símbolo: 8 bits : R0L, R0H, R1L, R1H
16 bits : R0, R1, R2, R3, A0, A1

Direccionamiento Indirecto a Registro: El valor de un registro de direcciones es la dirección efectiva para operar. El rango de direcciones efectiva es 00000_H a $0FFFF_H$.

Símbolo: [A0], [A1]

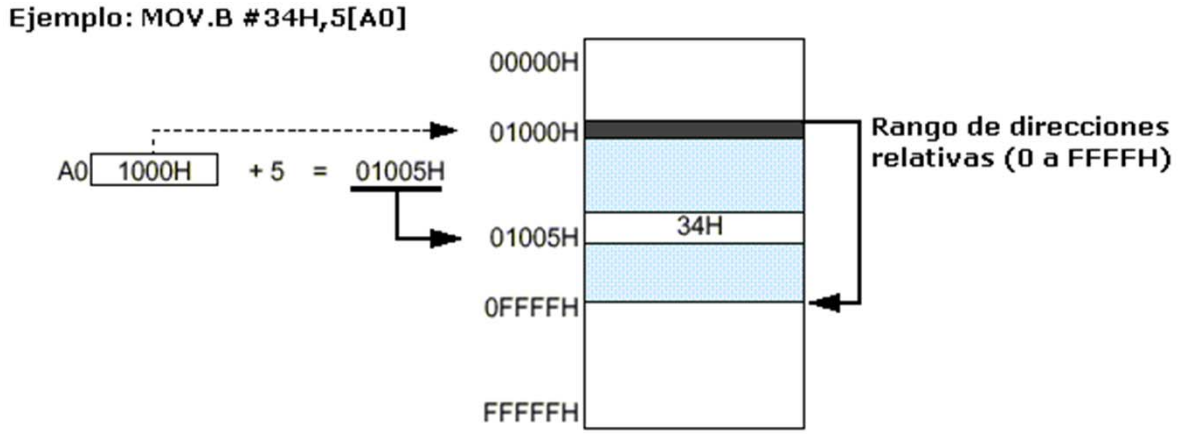
Ejemplo: MOV.B #12H, [A0]



3.3. Modos de direccionamiento

Direccionamiento Relativo a Registro: El valor de un registro más un desplazamiento (dsp) es la dirección efectiva con cuyo dato se va a realizar la operación. El rango de direccionamiento efectivo es 00000_H a $0FFFF_H$. Si el resultado de la suma excede de $0FFFF_H$, se ignara el bit más significativo.

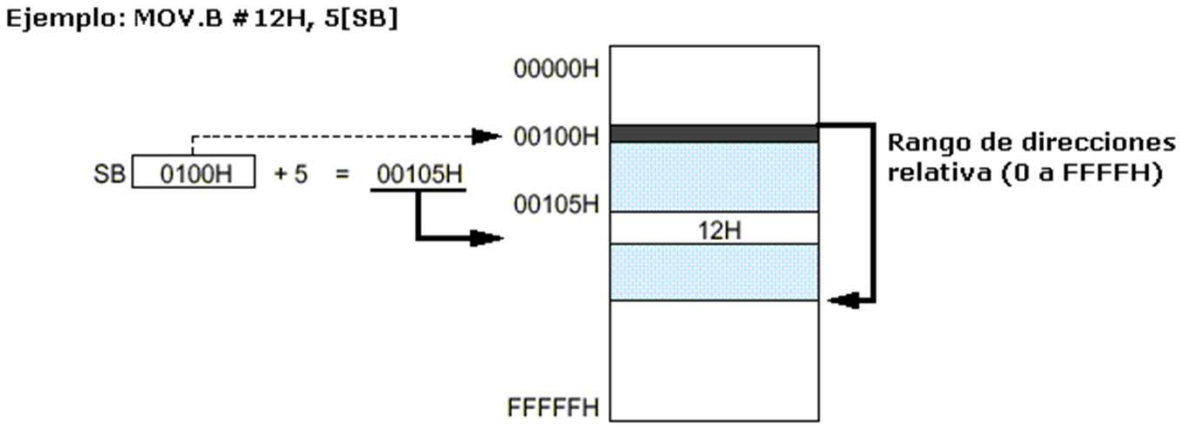
Símbolo: dsp[A0], dsp[A1]



3.3. Modos de direccionamiento

Relativo a SB: El valor del registro SB más un desplazamiento *dsp* es la dirección efectiva cuyo contenido se usará para operar. El rango de direccionamiento efectivo es 00000_H a $0FFFF_H$. Si el resultado de la suma excede $0FFFF_H$, el bit más significativo se ignora.

Símbolo: dsp[SB]



3.3. Modos de direccionamiento

Relativo a FB: El valor del registro FB más un desplazamiento *dsp* es la dirección efectiva que se usará para ser operada (figura 4.11). El rango de direcciones efectiva es 00000_H a $FFFF_H$.
Símbolo: dsp[FB]

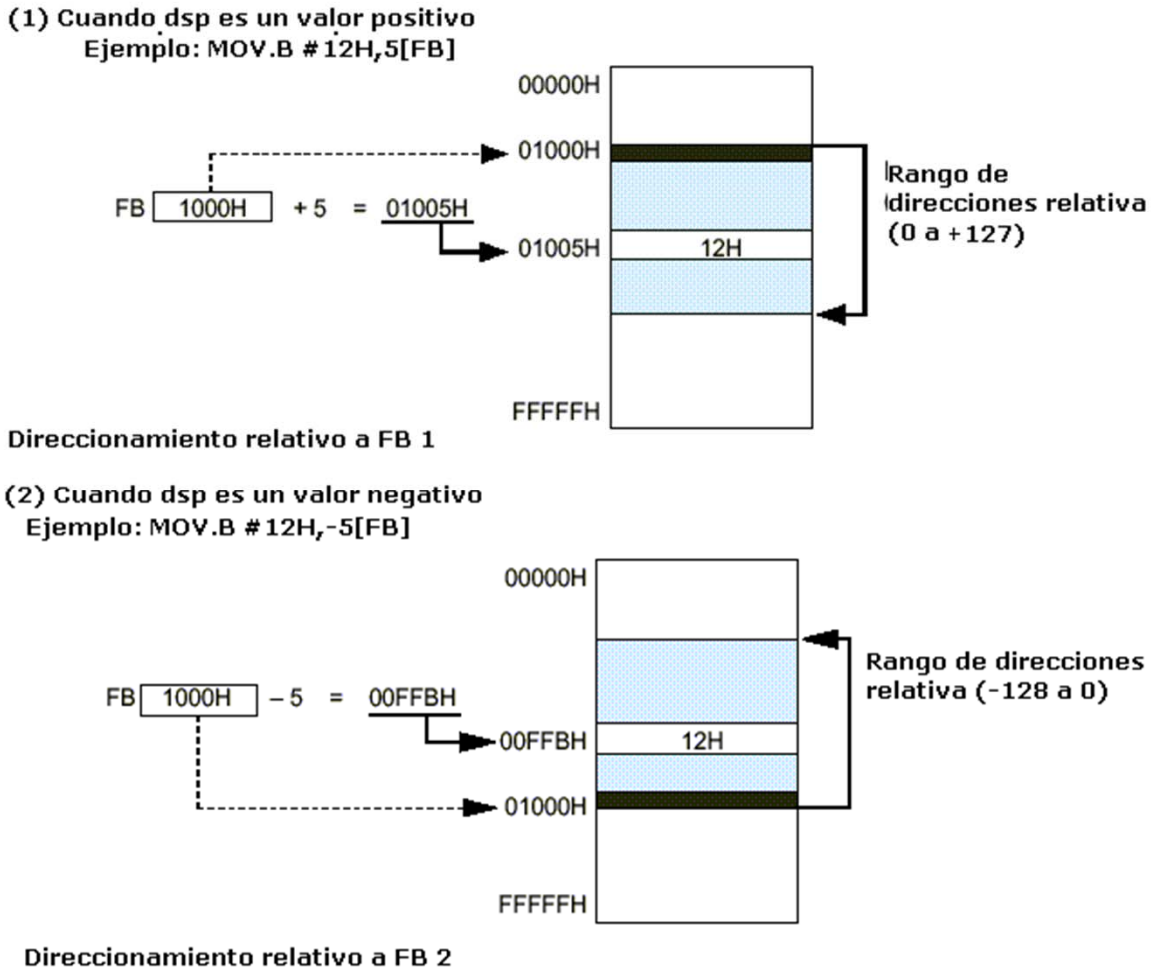


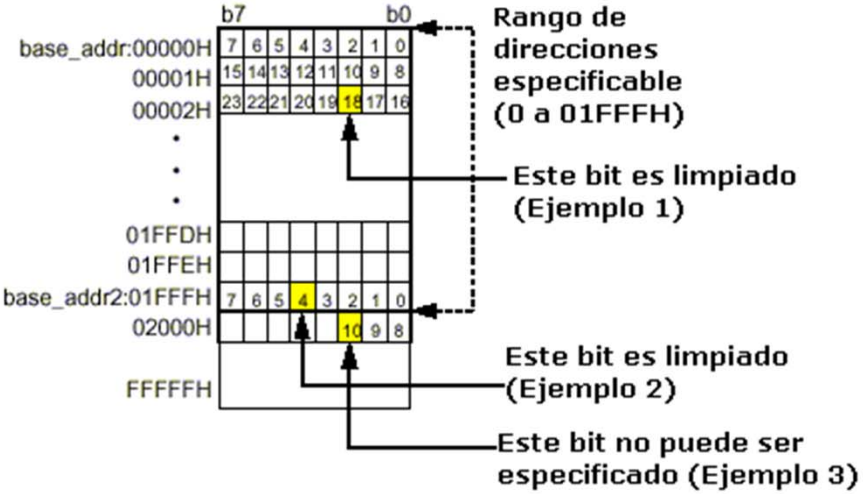
Figura 4.11. Direccionamiento Relativo a FB.

Relativo a SP (Puntero de pila)

3.3. Modos de direccionamiento

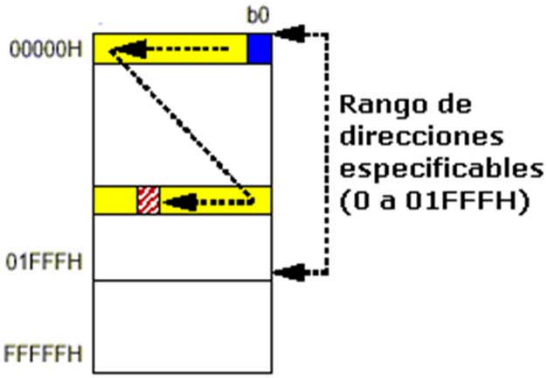
Absoluto de bit

Ejemplo 1: BCLR 18,base_addr
Ejemplo 2: BCLR 4,base_addr2
Ejemplo 3: BCLR 10,base-addr2 → Ejemplo 3 no puede ser especificado



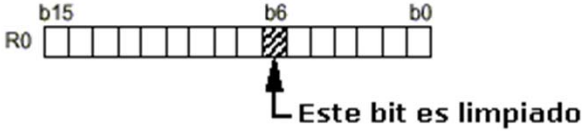
Indirecto a registro

Ejemplo: BCLR [A0]



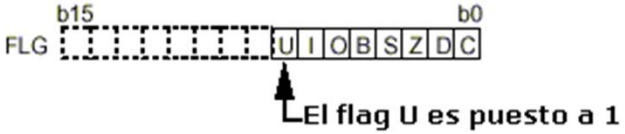
Directo a registro

Ejemplo: BCLR 6,R0



Directo a FGL

Ejemplo: FSET U



3.4. Gestión de direcciones

- La tabla de vectores fijos se usa para almacenar las direcciones de comienzo de subrutinas, las direcciones de las instrucciones de salto, JSRS y JMPS y las direcciones de las interrupciones no enmascarables.
- La tabla de vectores variables, se usa para indicar las direcciones de comienzo de las interrupciones de los periféricos de E/S y las de software.

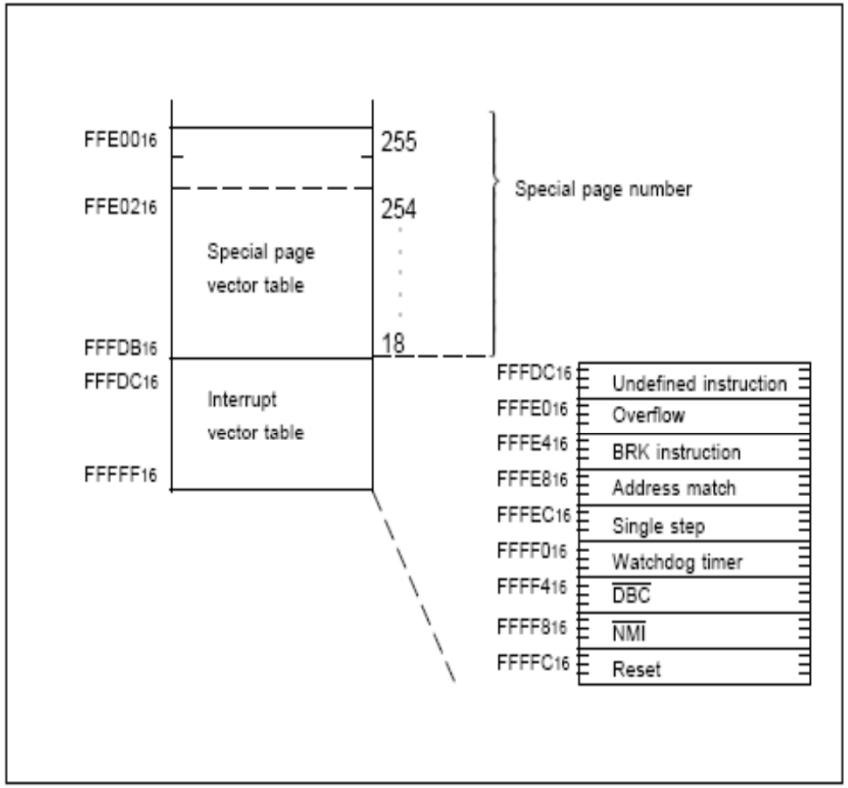


Figure 1.10.1 Fixed vector table

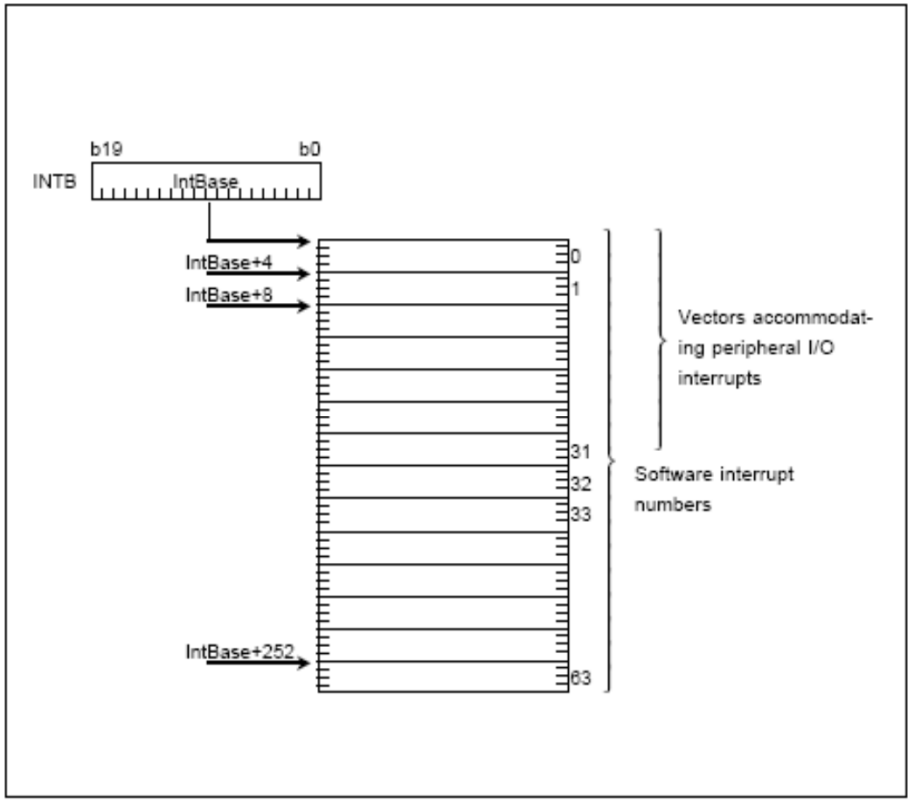


Figure 1.10.2 Variable vector table

3.4. Gestión de direcciones

Área de Vectores Fijos

El área de vectores fijos comprende el rango de direcciones desde la dirección FFE00H hasta la dirección FFFFFH. Consta de dos subáreas:

- **Tabla de vectores de página especial**. Comprende el rango de direcciones desde FFE00H hasta FFFDBH. Esta tabla es de tamaño fijo y se utiliza para almacenar las direcciones de comienzo de subrutinas y direcciones de salto, de tal forma que las llamadas a subrutinas y las instrucciones de salto se pueden ejecutar usando dos bytes, con la consiguiente reducción del número de pasos del programa. Cada tabla de vector está dividida en números de páginas especiales del 18 al 255 que contienen a su vez los 16 bits de bajo orden de la dirección de entrada de la subrutina, usadas por las instrucciones JSRS y JMPS.

- **Tabla de vectores de interrupción**. Comprende el rango de direcciones desde FFFDCH a la FFFFFH y ocupa 4 bytes por cada vector. Está área se reserva para las direcciones de comienzo de la rutina manejador de las interrupciónes hardware especiales: RESET, NMI, BRK, etc. (apartado 3.7.3). El programa monitor pone funciones *dummy* (*REIT*) en la tabla de vectores de interrupciones y las inhibe a todas, excepto a RESET, que se direcciona desde *Vvector .equ 0FFFDCH*. RESET es activa en el flanco de bajada (igual al 6800, el 8080 es actva en el flanco de subida), y es la encargada de ejecutar el programa monitor almacenado en la memoria flash, el cual contiene los datos, tablas y rutinas de trabajo para la ejecución de los programas de usuario. Se lleva a cabo en cuatro pasos:**1.** Para inmediatamente la ejecución en curso. **2**. Pone a cero todos los registros internos. **3**. El contador de programa se carga con los valores del vector del RESET, y **4**. Se ejecuta la subrutina asociada a la orden RESET. En el apartado 5.3 se muestra el código de inicialización del PC a través de la interrupción de RESET.

3.4. Gestión de direcciones

Área de Vectores Variables

Es el área de vectores de interrupciones de 256-bytes que usa el valor indicado en el registro de la tabla de interrupción (INTB) como la dirección efectiva de entrada (ver apartado 4.6 para conocer la dirección correcta de INT, la cual depende de cada sistema de desarrollo). Está compuesto de 4 bytes por tabla, así que son posibles 64 interrupciones: desde la 0 hasta la 31 para los vectores de interrupciones periféricas, y las restantes 32 para las interrupciones de E/S y por software, incluyendo la interrupción INT. A continuación, se muestra dicho listado, pero para ver cómo se direcciona es necesario leer los siguientes apartados y realizar la práctica del capítulo 5):

3.4. Gestión de direcciones

```
Tabla_Vector_Usuario:                    ; Dirección UART1 para MSA0652-G02
        .lword      dummy                        ; No0  Break Interrupt
        .lword      dummy                        ; No1  Break Interrupt
        .lword      dummy                        ; No2  Break Interrupt
        .lword      dummy                        ; No3  Break Interrupt
        .lword      dummy                        ; No4  Break Interrupt
        .lword      dummy                        ; No5  Break Interrupt
        .lword      dummy                        ; No6  Break Interrupt
        .lword      dummy                        ; No7  Break Interrupt
        .lword      dummy                        ; No8  Break Interrupt
        .lword      dummy                        ; No9  Break Interrupt
         .lword     dummy                        ; No10 Bus Clash Detect
        .lword      dummy                        ; No11 DMA
        .lword      dummy                        ; No12 DMA1
        .lword      dummy                        ; No13 KEY IN Interrupt
        .lword      dummy                        ; No14 A-D Interrupt
        .lword      dummy                        ; No15 UART2 Transmission Interrupt
        .lword      dummy                        ; No16 UART2 receive Interrupt
        .lword      dummy                        ; No17 UART0 Transmission Interrupt
        .lword      dummy                        ; No18 UART0 receive Interrupt
        .lword      0FCB6BH              ; No19 UART1 Transmission Interrupt
        .lword      0FCB6BH              ; No20 UART1 receive Interrupt
        .lword      sw_ta0                       ; No21 TimerA0 Interrupt
        .lword      dummy                ; No22 TimerA1 Interrupt
        .lword      dummy                        ; No23 TimerA2 Interrupt
        .lword      dummy                        ; No24 TimerA3 Interrupt
        .lword      dummy                        ; No25 TimerA4 Interrupt
        .lword      dummy                        ; No26 TimerB0 Interrupt
        .lword      dummy                        ; No27 TimerB1 Interrupt
        .lword      dummy                        ; No28 TimerB2 Interrupt
        .lword      dummy                        ; No29 INIT0(Active Low) Interrupt
        .lword      dummy                        ; No30 INIT1(Active Low) Interrupt
        .lword      dummy                        ; No31 INIT2(Active Low) Interrupt
```

3.4. Gestión de direcciones

```
•  ..lword  dummy  ; No32 S/W Interrupt
•  .lword   dummy  ; No33 S/W Interrupt
•  .lword   dummy  ; No34 S/W Interrupt
•  .lword   dummy  ; No35 S/W Interrupt
•  .lword   dummy  ; No36 S/W Interrupt
•  .lword   dummy  ; No37 S/W Interrupt
•  .lword   dummy  ; No38 S/W Interrupt
•  .lword   dummy  ; No39 S/W Interrupt
•  .lword   dummy  ; No40 S/W Interrupt
•  .lword   dummy  ; No41 S/W Interrupt
•  .lword   dummy  ; No42 S/W Interrupt
•  .lword   dummy  ; No43 S/W Interrupt
•  .lword   dummy  ; No44 S/W Interrupt
•  .lword   dummy  ; No45 S/W Interrupt
•  .lword   dummy  ; No46 S/W Interrupt
•  .lword   dummy  ; No47 S/W Interrupt
•  .lword   dummy  ; No48 S/W Interrupt
•  .lword   dummy  ; No49 S/W Interrupt
•  .lword   dummy  ; No50 S/W Interrupt
•  .lword   dummy  ; No51 S/W Interrupt
•  .lword   dummy  ; No52 S/W Interrupt
•  .lword   dummy  ; No53 S/W Interrupt
•  .lword   dummy  ; No54 S/W Interrupt
•  .lword   dummy  ; No55 S/W Interrupt
•  .lword   dummy  ; No56 S/W Interrupt
•  .lword   dummy  ; No57 S/W Interrupt
•  .lword   dummy  ; No58 S/W Interrupt
•  .lword   dummy  ; No59 S/W Interrupt
•  .lword   dummy  ; No60 S/W Interrupt
•  .lword   dummy  ; No61 S/W Interrupt
•  .lword   dummy  ; No62 S/W Interrupt
•  .lword   dummy  ; No63 S/W Interrupt
```

3.5. Directivas y funciones macro

- Las directivas del ensamblador, también llamadas pseudos-instrucciones o pseudos-operaciones, son órdenes dadas al programa ensamblador que no son traducidas al código máquina. No afectan al programa fuente, pero son totalmente imprescindibles para la compilación. Su objetivo es el control de la ejecución del programa ensamblador.

- Atendiendo a la función que realizan, se les clasifica en:
- Directivas para el control de módulos.
- Directivas para el control de direcciones.
- Directivas para asignamiento de valores
- Directivas para el control del ensamblador.
- Directivas para el control de enlaces (*link*).
- Directivas para el control de listados.
- Directiva para el control óptimo de las bifurcaciones.
- Directivas para el ensamblado condicional.
- Directivas para funciones extendidas.

3.5. Directivas y funciones macro

Directivas para el control de módulos

Se utilizan para fijar el comienzo y el final de los distintos módulos del programa fuente y asignarles nombres y tipos. Están disponibles las siguientes:

- **.NAME (.PROGRAM)** Define el comienzo de un módulo del programa.
- **.MODULE (.LIBRARY)** Define el comienzo de un módulo de librería.
- **.ENDMOD** Define el final del ensamblado del módulo actual
- **.END** Define el final del ensamblado del último módulo del fichero, es decir, indica que se ha terminado la traducción a código máquina del programa.

Directivas para el control de direcciones

Se utilizan para determinar la dirección en la que se ensambla el programa o bloques del programa. Están disponibles las siguientes:

- **.ORG** (ORiGen). Indica al programa ensamblador en qué posición de la memoria deberá comenzar a almacenar los códigos máquinas de las instrucciones del programa.
  Ejemplo: .ORG 0F0000H; .ORG VRamTOP+offset
- **.BLKx** Asigna un área de la memoria RAM en unidades de *x* Bytes. Si x=B, se asigna 1 byte; Si x=W, se asignan 2 bytes; Si x=A, se asigna 3 bytes; Si x=L, se asignan 4 bytes; Si x=F, se asigna 4 bytes para números en punto flotante; Si x=D, se asigna 8 bytes.
  Ejemplo: .BLKB 1; .BLKA number +1; label: .BLKF number
- **.BYTE** Almacena datos en la memoria ROM en longitud de 1 byte.
- **.WORD** Almacena datos en la memoria ROM en longitud de 2 bytes.
- **.ADDR** Almacena datos en la memoria ROM en longitud de 3 bytes.
- **.LWORD** Almacena datos en la memoria ROM en longitud de 4 bytes para los números en punto flotante.
- **.FLOAT** Almacena un número en punto flotante en la memoria ROM en longitud de 4 bytes.
- **.DOUBLE** Almacena un número en punto flotante en la memoria ROM en longitud de 8 bytes.
  Ejemplo: .BYTE 1,2; .FLOAT 5E2; .ADDR símbolo;
- **.ALIGN** Corrige direcciones impares a direcciones pares.

3.5. Directivas y funciones macro

Address control	
.ORG	Declares address.
.BLKB	Allocates RAM area in units of 1 bytes.
.BLKW	Allocates RAM area in units of 2 bytes.
.BLKA	Allocates RAM area in units of 3 bytes.
.BLKL	Allocates RAM area in units of 4 bytes.
.BLKF	Allocates RAM area in units of 4 bytes.
.BLKD	Allocates RAM area in units of 8 bytes.
.BYTE	Stores data in ROM in 1-byte length.
.WORD	Stores data in ROM in 2-byte length.
.ADDR	Stores data in ROM in 3-byte length.
.LWORD	Stores data in ROM in 4-byte length.
.FLOAT	Stores data in ROM in 4-byte length.
.DOUBLE	Stores data in ROM in 8-byte length.
ALIGN	Corrects odd addresses to even addresses.

Assemble control	
.EQU	Defines symbol.
.BTEQU	Defines bit symbol.
.END	Declares end of assemble source.
.SB	Assigns temporary SB register value.
.SBSYM	Selects SB relative displacement addressing mode.
.SBBIT	Selects SB relative displacement addressing mode for bit symbol.
.FB	Assigns temporary FB register value.
.FBSYM	Selects FB relative displacement addressing mode.
.INCLUDE	Reads file into specified position.

3.5. Directivas y funciones macro

Directivas para el control del ensamblador

Se utilizan para dirigir la ejecución del ensamblador AS30 o A30. Están disponibles las siguientes:

- **.EQU** Define un símbolo, es decir, asigna un valor constante sólo para el módulo en el que se declara. Ejemplo: valor1 .EQU 1
- **.BTEQU** Define un símbolo a nivel de bit. Ejemplo: bit3 .EQU 0,0
- **.END** Define el final del ensamblado del último módulo del fichero, es decir, indica que ha terminado la traducción a código máquina del programa, ignorando el resto del código. Debe existir al menos una directiva pro fichero fuente.
- **.SB** Asume un valor en el registro SB. Ejemplo: .SB 80H
- **.FB** Asume un valor en el registro SB. Ejemplo: .FB 0C0H
- **.SBSYM** Elige el direccionamiento relativo SB.
- **.SBBIT** Elige el direccionamiento relativo SB para instrucciones de bits.
- **.FBSYM** Elige el direccionamiento relativo FB.
- **.INCLUDE** Lee un fichero en una posición especifica.
  Ejemplos: .INCLUDE sfr.inc .INCLUDE parpadeo.a30

3.5. Directivas y funciones macro

Link control	
.SECTION	Defines section name.
.GLB	Specifies global label.
.BTGLB	Specifies global bit symbol.
.VER	Transfers specified information to map file.

List control	
.LIST	Controls outputting of line data to list file.
.PAGE	Breaks page at specified position of list file.
.FORM	Specifies number of columns and lines in 1 page of list file.

Branch instruction optimization control	
.OPTJ	Controls optimization of branch instruction and subroutine call instruction.

Extended Function Directive Commands	
.ASSERT	Outputs a character string written in the operand to a standard error output device or file.
?	Specifies defining and referencing a temporary label.
..FILE	Indicates the assembly source file name being processed by as30.
@	Concatenates character strings entered before and after @ into a single character string.

Conditional Assemble Control	
.IF	Indicates the beginning of a conditional assemble block. Conditions are resolved.
.ELIF	Resolves the second and the following conditions.
.ELSE	Indicates the beginning of a block to be assembled.
.ENDIF	Indicates the end of a conditional assemble block.

3.5. Directivas y funciones macro

Directivas para el control de enlaces

Se utilizan para definir información para la reasignación de las direcciones de control.

.VER Es la salida a un fichero **.map* de una cadena de caracters específica. Sólo se puede escribir una vez en un fichero fuente. Ejemplo: .VER 'cadena'

.GLB Especifica una etiqueta global.

Ejemplo: .GLB nombre1,nombre2

.BTGLB Especifica un símbolo global para un bit.

Ejemplo: .BTGLB flag2

.SECTION Define un nombre de sección. Existen tres tipos de sección:

CODE área donde se escribe el programa (ROM).
- Ejemplo: .SECTION miprograma, CODE

DATA área de datos (RAM).
- Ejemplo: .SECTION misdatos, DATA

ROMDATA área de datos fijada en la memoria ROM.
- Ejemplo: .SECTION constantes, CODE

Directivas para el control de listados

Se utilizan para controlar el formato de los ficheros de listado generados por el ensamblador AS30 o A30.

.LIST Controla las líneas que se van a volcar al fichero de listado. OFF para no volcar lo que viene a continuación y ON para comenzar el listado. Por defecto está en ON.

Ejemplo: .LIST OFF; .LIST ON

.PAGE Hace una rupturta de la página en la posición especificada en el fichero de listado.

Ejemplo: .PAGE 'string'

.FORM Especifica un número de líneas y de columnas en una página para el formato del fichero de listado. Por defecto es 66 líneas y 140 columnas

Ejemplo: .FORM 20,80; 20 líneas y 80 columnas

3.5. Directivas y funciones macro

Macro directive commands	
.MACRO	Defines macro name.Indicates beginning of macro body.
.EXITM	Stops expansion of macro body.
.LOCAL	Declares local label in macro.
.ENDM	Indicates end of macro body.
.MREPEAT	Indicates beginning of repeat macro body.
.ENDR	Indicates end of repeat macro body.
Macro symbols	
..MACPARA	Indicates number of actual parameter of macro call.
..MACREP	Indicates how many times repeat macro body is expanded.
Character string functions	
.LEN	Indicates length of specified character string.
.INSTR	Indicates start position of specified character string in specified character string.
.SUBSTR	Extracts specified number of characters from specified character string beginning with specified position.

3.5. Directivas y funciones macro

Directivas para el ensamblado condicional

Se utilizan para que en el ensamblado exista un bloque en el cual el código se ha generado de acuerdo a unas condiciones específicas. Están disponibles las siguientes:

.IF	Indica el comienzo del ensamblado condicional.
.ELIF	Indica la condición para el ensamblado condicional.
.ELSE	Indica el comienzo del bloque a ensamblar cuando la condición es falsa.
.ENDIF	Indica el final del ensamblado condicional.

```
Ejemplo:    .IF    cond==0
                        .BYTE “modo1”
                                .ELIF  cond>0
                        .BYTE “modo2”
                                .ELSE
                        .BYTE “modo3”
            .ENDIF
```

Hay que tener cuidado puesto que el ensamblador no detecta los posibles *overflows* o *underflows.*

También, existen estamentos para descripción estructuradas con las más usuales de cualquier lenguaje de programación:

FOR NEXT
FOR TO STEP NEXT
DO WHILE
SWITCH CASE DEFAULT ENDS
BREAK
CONTINUE
FOREVER

3.5. Directivas y funciones macro

Funciones Macro

•Las funciones macro se definen con la directiva **.MACRO** y contienen un conjunto de instrucciones hasta que termina con la directiva **.ENDM**. Se pueden definir etiquetas locales declaradas con la directiva **.LOCAL.** Las llamadas a la función macro se realizan escribiendo el nombre que se le asignó en la definición .MACRO. La directiva **.EXITM** indica que se debe salir de la ejecución de la macro.

•También, se puede definir una función macro que se realice un número determinado de veces. Para ello, se usa la directiva .MREPEAT, terminando con la directiva .ENDR y pasándole un argumento que indica el número de veces que se debe repetir dicha macro.

3.6. Ejemplo de programa fuente

```
;***********************************************
;                          Programa
;***********************************************
;
;----- Fichero sfr-----------------------------------
;
            .list  off                  ; deshabilitar listado
            .include   sfr62.inc        ; incluir fichero de símbolos y direcciones SFR
            .list  on                   ; habilitar listado
;
;----- Definición de símbolos   -------  Placa 3DKM16C/62PU ----------------
;
VramTOP     .equ        000400h     ; direccion superior RAM
VramEND     .equ        007D00h     ; direccion superior RAM
VIstack     .equ        007CFFh     ; puntero de pila
VprogTOP    .equ        0A0000h     ; direccion superior ROM
Vintbase    .equ        0FA000h     ; direccion superior de la tabla de vectores variables
Vvector     .equ        0FFFDCh     ; direccion superior de la tabla de vectores fija
SB_base     .equ        000380h     ; direccion base SB
;
```

3.6. Ejemplo de programa fuente

```
;----- Área de datos RAM ----------------------------------------
;
                .section        memory, data
                .org            VramTOP
;
;----- Área de código ROM/Flash ---------------------------------
;
                .section        prog, code
                .org            VprogTOP
;
;----- Limpieza de la RAM  --------------------------------
;
                mov.w           #0, R0                          ; 0 clear
                mov.w           #(VramEND+1-VramTOP)/2,R3       ; number of times
                mov.w           #VramTOP, A1                    ; start address
                sstr.w
;
;----- Iinicialización ------------------------------------
;
                bset            PD8_0                           ; P8_0 output
                mov.b           #80h, TA0MR                     ; set TA0MR
                mov.w           #625, TA0                       ; make 2msec
                bset            TA0S                            ; TA0 start
;
;----- Rutina principal -----------------------------
;
main:
                mov.w           #0, r0                          ; r0 clear
check:
                btstc           3,TA0IC                         ; 2msec?
                jnc             check
                add.w           #1,r0                           ; r0 count up
                cmp.w           #500,r0                         ; r0=500?
                jnz     check
                bnot            P8_0                            ; turn on & off
                jmp             main
;
;----- Fin del programa ----------------------------
                .end
```